

Experiment No. 5

Aim

To implement the Round Robin CPU Scheduling Algorithm and evaluate the average turnaround time and average waiting time for a given set of processes.

Theory

Round Robin (RR) is a preemptive CPU scheduling algorithm designed primarily for time-sharing systems, where fairness and responsiveness matter as much as throughput. Each process in the ready queue is assigned a fixed unit of CPU time called a time quantum (or time slice). Processes are held in a circular queue: the process at the front of the queue is given the CPU for at most one time quantum. If the process completes within that time quantum, it leaves the system; otherwise, it is preempted and placed at the back of the ready queue, and the CPU moves on to the next process in the queue.

Unlike SJF or SRTF, Round Robin does not require any knowledge of a process's burst time in advance to make scheduling decisions - every process is treated equally and given the CPU in a fixed cyclic order, which makes it simple to implement and inherently fair, with no process ever starving indefinitely. The performance of Round Robin is highly sensitive to the choice of time quantum: a very large time quantum makes RR behave like non-preemptive FCFS (poor responsiveness for short jobs), while a very small time quantum increases fairness and responsiveness but adds significant context-switching overhead, since the CPU has to save and restore process state far more frequently.

For each process, the key performance metrics are computed as:

- Completion Time (CT): the time at which a process finally finishes execution, after possibly running for multiple time quanta across several turns in the queue.
- Turnaround Time (TAT): $TAT = Completion\ Time - Arrival\ Time$, the total time from arrival to completion.
- Waiting Time (WT): $WT = Turnaround\ Time - Burst\ Time$, the cumulative time spent waiting in the ready queue across all the turns before completion.

The scheduling proceeds by repeatedly dequeuing the process at the front of the ready queue, executing it for $\min(\text{remaining burst time}, \text{time quantum})$ units, updating its remaining burst time, and - if it is not yet finished - re-adding it to the back of the queue (after first adding any processes that arrived during this time slice, as per standard convention). This continues until all processes have completed, after which the average turnaround time and average waiting time are computed by summing these values across all processes and dividing by the total number of processes.

Code

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <string>
using namespace std;
```

```

struct Process {
    int id;
    int at;
    int bt;
    int ct;
    int tat;
    int wt;
};

//1. Structure to record Gantt chart timeline blocks
struct GanttRecord {
    int id;
    int start_time;
    int end_time;
};

int main() {
    int n;
    cout << "Enter number of processes: ";
    cin >> n;
    int tq;
    cout << "Enter Time Quantum: ";
    cin >> tq;

    int burstSum = 0;
    vector<Process> p(n);
    vector<int> rem_bt(n);

    for (int i = 0; i < n; ++i) {
        p[i].id = i + 1;
        cout << "Enter Arrival Time for process " << p[i].id << ": ";
        cin >> p[i].at;
        cout << "Enter Burst Time for process " << p[i].id << ": ";
        cin >> p[i].bt;
        rem_bt[i] = p[i].bt;
        burstSum += p[i].bt;
        p[i].ct = p[i].tat = p[i].wt = 0;
    }

    sort(p.begin(), p.end(), [](const Process& a, const Process& b) {
        return a.at < b.at;
    });

    int currentTime = p[0].at;
    vector<GanttRecord> gantt; //Vector to store execution history

    while(burstSum > 0) {
        bool idle = true;

        for (int i = 0; i < n; ++i) {

```

```

if (p[i].at <= currentTime && rem_bt[i] > 0) {
    idle = false;
    int start = currentTime; //Save block start point

    int executionTime = min(tq, rem_bt[i]);
    rem_bt[i] -= executionTime;
    burstSum -= executionTime;
    currentTime += executionTime;

    //2. Record this execution interval into the timeline
    gantt.push_back({p[i].id, start, currentTime});

    if (rem_bt[i] == 0) {
        p[i].ct = currentTime;
        p[i].tat = p[i].ct - p[i].at;
        p[i].wt = p[i].tat - p[i].bt;
    }
}
}
if (idle) {
    //Track idle CPU time in the Gantt chart as Process ID: -1
    int start = currentTime;
    currentTime++;
    gantt.push_back({-1, start, currentTime});
}
}

// Display results table
cout << "\nProcess\tArrival Time\tBurst Time\tCompletion Time\tTurnaround
Time\tWaiting Time\n";
for (int i = 0; i < n; ++i) {
    cout << p[i].id << "\t" << p[i].at << "\t\t" << p[i].bt << "\t\t" << p[i].ct << "\t\t" <<
p[i].tat << "\t\t" << p[i].wt << "\n";
}

// Calculate and display average TAT and WT
double totalTAT = 0, totalWT = 0;
for (int i = 0; i < n; ++i) {
    totalTAT += p[i].tat;
    totalWT += p[i].wt;
}
cout << "\nAverage Turnaround Time: " << totalTAT / n;
cout << "\nAverage Waiting Time: " << totalWT / n << "\n";

// 3. Print the Gantt Chart blocks
cout << "\n--- GANTT CHART---\n\n ";
for (const auto& block : gantt) {
    if (block.id == -1) cout << "| Idle ";
    else cout << "| P" << block.id << " ";
}

```

```

cout << "\\n ";

// Print the timeline markers matching the columns
for (const auto& block : gantt) {
    cout << block.start_time;
    if (block.start_time < 10) cout << "   ";
    else cout << "   ";
}
cout << gantt.back().end_time << "\\n";

return 0;
}

```

Output

```

PS C:\Users\lenovo\Desktop\Development\OS Programs> cd "c:\Users\lenovo\Desktop\Development\OS Programs\" ; if ($?) { g++ rr.cpp -o rr } ; if ($?) { .\rr }
Enter number of processes: 4
Enter Time Quantum: 2
Enter Arrival Time for process 1: 0
Enter Burst Time for process 1: 4
Enter Arrival Time for process 2: 2
Enter Burst Time for process 2: 5
Enter Arrival Time for process 3: 3
Enter Burst Time for process 3: 6
Enter Arrival Time for process 4: 5
Enter Burst Time for process 4: 7

Process Arrival Time    Burst Time    Completion Time Turnaround Time Waiting Time
1         0             4             10             10             6
2         2             5             17             15             10
3         3             6             19             16             10
4         5             7             22             17             10

Average Turnaround Time: 14.5
Average Waiting Time: 9

--- GANTT CHART---

| P1 | P2 | P3 | P4 | P1 | P2 | P3 | P4 | P2 | P3 | P4 | P4 |
0   2   4   6   8  10  12  14  16  17  19  21  22

```

Viva Questions

Q1. What is a time quantum in Round Robin scheduling, and how does its size affect performance?

Ans. A time quantum (or time slice) is the fixed maximum amount of CPU time given to a process before it is preempted and moved to the back of the ready queue. A very large time quantum makes Round Robin behave similarly to non-preemptive FCFS, reducing responsiveness for short processes. A very small time quantum improves fairness and responsiveness but increases the frequency of context switches, adding significant overhead.

Q2. Why is Round Robin considered fairer than SJF or SRTF?

Ans. Round Robin gives every process an equal, fixed opportunity to use the CPU in a fixed cyclic order, regardless of its burst time, so no process can be indefinitely delayed by a continuous stream of other processes as can happen in SJF or SRTF. This guarantees that every process makes progress and none of them starves.

Q3. Does Round Robin require prior knowledge of a process's burst time? How does this differ from SJF/SRTF?

Ans. No, Round Robin does not require any advance knowledge of burst times to make scheduling decisions — it simply cycles through all processes in the ready queue, giving each a fixed time quantum. In contrast, SJF and SRTF must know (or estimate) burst times or remaining times in advance in order to always select the shortest job.

Q4. What happens when a process's remaining burst time is smaller than the time quantum?

Ans. If a process's remaining burst time is less than or equal to the time quantum, it runs for exactly its remaining time (not the full quantum), completes, and leaves the ready queue rather than being preempted and re-queued. This ensures the CPU is not left idle for the unused portion of the time quantum.

Q5. What is the main drawback of using a very small time quantum in Round Robin?

Ans. Using a very small time quantum causes the CPU to switch between processes extremely frequently, and each of these context switches carries real overhead — saving and restoring process state/registers. If the time quantum becomes too small relative to this switching overhead, a large fraction of CPU time is wasted on context switching rather than actual process execution, reducing overall system throughput.